LAPORAN TUGAS KELOMPOK

Mata Kuliah Sistem Operasi (USA-WP2260110)

Simulasi dan Evaluasi Algoritma Penjadwalan CPU pada Gateway Kebun Pintar

Keterangan	Isi
Kode dan judul tugas	K-2 Simulasi dan Evaluasi Algoritma Penjadwalan CPU
Kelas / tema	IF-3C — IoT dan Smart Farming
Nama	Dendy Zainal Mutaqien
NIM	250660121068
Anggota kelompok	1 orang (rincian di Lampiran D)
Nama host	INFINIX-ElementaryOS
Minggu / tanggal kumpul	Minggu ke-5 / 5 November 2026
Kepingan puzzle	Kepingan #5 → Level 3 Puzzle UTS
Dosen pengampu	Muhammad Agreindra Helmiawan, S.Kom., M.T

Program Studi S1 Informatika
Fakultas Teknologi Informasi · Universitas Sebelas April
2026

Daftar Isi

1. Pendahuluan..... 3
1.1 Latar belakang kasus..... 3
1.2 Tujuan tugas..... 3
1.3 Ruang lingkup..... 3
2. Dasar Teori Singkat..... 3
3. Lingkungan Praktik..... 4
4. Langkah Pengerjaan..... 4
4.1 Langkah 1: Menulis simulator dengan satu mesin waktu..... 4
4.2 Langkah 2: Menghitung manual sebagai pembanding..... 7
4.3 Langkah 3: Membuat Gantt chart..... 7
5. Hasil dan Analisis..... 9
5.1 Hasil..... 9
5.2 Analisis..... 10
5.3 Rekomendasi..... 11
6. Kesimpulan..... 11
7. Kepingan Puzzle..... 12
Daftar Pustaka..... 12
Lampiran A. Kode Program dan Log..... 13
Lampiran B. Pernyataan Penggunaan AI Generatif..... 16
Lampiran C. Daftar Periksa Sebelum Mengumpulkan..... 16
Swa-nilai berdasarkan kriteria lembar tugas..... 17
Lampiran D. Lembar Kontribusi Anggota..... 18

1. Pendahuluan

1.1 Latar belakang kasus

Gateway kebun pintar menjalankan beberapa tugas sekaligus pada satu perangkat kecil: membaca sensor, mengendalikan pompa irigasi, dan mengirim data ke server. Semua tugas itu bersaing memperebutkan satu prosesor. Kalau penjadwalannya buruk, pembacaan sensor bisa tertunda beberapa detik, atau perintah menutup pompa baru dijalankan setelah air sudah meluap.

Penjadwalan CPU menentukan urutan dan lama setiap tugas memakai prosesor. Algoritma yang berbeda menghasilkan rata-rata waktu tunggu, waktu penyelesaian, dan waktu respons yang berbeda pula. Pada perangkat yang tugasnya bercampur antara pekerjaan panjang dan pendek, pilihan algoritma bukan soal selera: ia menentukan apakah tugas mendesak benar-benar dilayani lebih dulu. Tugas ini membandingkan enam konfigurasi penjadwalan pada satu dataset kelas yang sama, memakai simulator yang hasilnya diverifikasi dengan perhitungan manual.

1.2 Tujuan tugas

Capaian pembelajaran yang diukur adalah Sub-CPMK-5 (CPMK061, CPMK063, CPMK081, CPMK102, CPMK112). Tujuannya:

- **Pertama,** membuat Gantt chart dan menghitung rata-rata waiting time, turnaround time, dan response time untuk FCFS, SJF, SRTF, Priority, serta Round Robin pada quantum 2 dan 3;
- **Kedua,** membangun simulator penjadwalan sederhana, lalu memverifikasi keluarannya terhadap perhitungan manual;
- **Ketiga,** merekomendasikan algoritma yang paling sesuai untuk konteks gateway kebun pintar.

1.3 Ruang lingkup

Dataset kelas yang dipakai adalah P1(0,3,1), P2(0,6,3), P3(2,4,2), P4(3,8,4), dan P5(4,2,1) dengan format (arrival, burst, prioritas). Angka prioritas kecil berarti prioritas lebih tinggi. Simulator memakai satuan waktu satu tick dan mengasumsikan waktu konteks-switch nol. Algoritma preemptive yang diuji adalah SRTF dan Round Robin; algoritma non-preemptive yang diuji adalah FCFS, SJF, dan Priority.

Yang tidak dikerjakan: pengukuran pada perangkat keras nyata, penjadwalan multiprosesor, dan analisis pengaruh waktu konteks-switch.

2. Dasar Teori Singkat

Penjadwal CPU memilih proses mana yang berjalan saat prosesor bebas. Silberschatz dkk. (2018) mengelompokkan kriteria penilaiannya menjadi waktu tunggu (waiting time), waktu penyelesaian (turnaround time), dan waktu respons (response time). Waiting time adalah total waktu proses berada di antrean siap; turnaround time adalah selisih antara waktu selesai dan waktu kedatangan; response time adalah selisih antara waktu pertama kali

mendapat prosesor dan waktu kedatangan.

FCFS (First Come First Served) melayani proses sesuai urutan kedatangan. Algoritma ini sederhana dan tidak preemptive, tetapi terkenal dengan efek konvoi: satu proses panjang di depan menahan semua proses di belakangnya. SJF (Shortest Job First) memilih proses dengan burst time terkecil. SJF terbukti memberi rata-rata waiting time minimum untuk himpunan proses yang sudah tersedia, tetapi versi non-preemptive-nya merugikan proses yang datang lebih dulu. SRTF (Shortest Remaining Time First) adalah versi preemptive dari SJF: setiap kali ada proses baru datang, sisa waktu proses yang sedang berjalan dibandingkan, dan prosesor berpindah ke yang sisanya lebih pendek.

Priority scheduling memilih proses dengan prioritas tertinggi. Tanpa aging, algoritma ini dapat membuat proses berprioritas rendah tidak pernah dilayani (starvation). Round Robin menghapus kelemahan itu dengan memberi setiap proses jatah waktu tetap (quantum) secara bergiliran. Quantum kecil membuat response time rendah tetapi menambah jumlah pergantian konteks; quantum besar mendekati perilaku FCFS. Pada Round Robin, besarnya quantum menjadi parameter yang harus dipilih, bukan sesuatu yang datang sendiri.

3. Lingkungan Praktik

Komponen	Keterangan
Laptop (host)	elementary OS 8, AMD Ryzen 7 7735HS, RAM 16 GiB, kernel 7.0.0-34-generic
Hypervisor	Tidak dipakai — simulasi dijalankan langsung pada host
VM / host	INFINIX-ElementaryOS, 16 vCPU
Perangkat lain	Tidak berlaku — dataset kelas dipakai sebagai beban simulasi
Bahasa dan alat	Python 3.12.3; matplotlib untuk Gantt chart dan grafik; python-docx untuk laporan
Satuan waktu	1 tick; asumsi waktu konteks-switch nol

4. Langkah Pengerjaan

Pekerjaan dibagi menjadi tiga tahap: menulis simulator, menghitung manual sebagai pembanding, lalu membuat Gantt chart. Folder kerja berisi sched.py (simulator), figures.py (gambar), dan results/ (keluaran JSON dan PNG).

4.1 Langkah 1: Menulis simulator dengan satu mesin waktu

Agar FCFS, SJF, Priority, SRTF, dan Round Robin bisa dibandingkan dengan aturan yang sama, simulator tidak ditulis sebagai lima fungsi terpisah. Semua algoritma memakai satu loop waktu yang naik satu tick, dan perbedaannya hanya pada fungsi pemilih proses. Round Robin ditangani dengan menghitung tick yang sudah dipakai proses berjalan; SRTF dengan membandingkan sisa waktu setiap kali ada proses baru datang.

```
"""Simulator penjadwalan CPU untuk dataset kelas K-2.

Dataset: P(arrival, burst, prioritas). Angka prioritas kecil = prioritas tinggi.
Simulasi memakai satuan waktu 1 tick supaya algoritma preemptive (SRTF, RR)
dapat dibandingkan dengan aturan yang sama.
"""
import argparse
import json
```

```
# name, arrival, burst, priority
DATASET = [
    ("P1", 0, 3, 1),
    ("P2", 0, 6, 3),
    ("P3", 2, 4, 2),
    ("P4", 3, 8, 4),
    ("P5", 4, 2, 1),
]


def simulasi(kind, quantum=None, data=DATASET):
    """Kembalikan (segmen, metrik). segmen = [(nama, mulai, selesai), ...]."""
    arr = {n: a for n, a, b, p in data}
    burst = {n: b for n, a, b, p in data}
    prio = {n: p for n, a, b, p in data}
    sisa = dict(burst)
    urut = [n for n, *_ in data]

    menunggu = []
    selesai = set()
    mulai_pertama = {}
    waktu_selesai = {}
    segmen = []

    def catat(nama, t):
        if segmen and segmen[-1][0] == nama and segmen[-1][2] == t:
            segmen[-1] = (nama, segmen[-1][1], t + 1)
        else:
            segmen.append((nama, t, t + 1))

    def pilih(kandidat, aturan):
        if aturan == "fcfs":
            return min(kandidat, key=lambda n: (arr[n], urut.index(n)))
        if aturan == "sjf":
            return min(kandidat, key=lambda n: (burst[n], arr[n], urut.index(n)))
        if aturan == "srtf":
            return min(kandidat, key=lambda n: (sisa[n], arr[n], urut.index(n)))
        if aturan == "priority":
            return min(kandidat, key=lambda n: (prio[n], arr[n], urut.index(n)))
        raise ValueError(aturan)

    preemptive = kind in ("srtf", "rr")
    aturan = "fcfs" if kind == "rr" else kind

    t = 0
    sekarang = None
    terpakai = 0

    while len(selesai) < len(data):
        for n in urut:
            if arr[n] == t:
                menunggu.append(n)

        if sekarang is None:
            if not menunggu:
                t += 1
                continue
            sekarang = pilih(menunggu, aturan) if kind != "rr" else menunggu.pop(0)
            if kind != "rr":
                menunggu.remove(sekarang)
            terpakai = 0
        elif kind == "srtf":
            kandidat = pilih(menunggu, "srtf") if menunggu else None
            if kandidat and sisa[kandidat] < sisa[sekarang]:
                menunggu.remove(kandidat)
                menunggu.append(sekarang)
                sekarang = kandidat
                terpakai = 0
        elif kind == "rr" and terpakai == quantum:
            menunggu.append(sekarang)
            sekarang = menunggu.pop(0)
            terpakai = 0

        mulai_pertama.setdefault(sekarang, t)
```

```
        catat(sekarang, t)
        sisa[sekarang] -= 1
        terpakai += 1
        t += 1

        if sisa[sekarang] == 0:
            waktu_selesai[sekarang] = t
            selesai.add(sekarang)
            sekarang = None
            terpakai = 0

    metrik = {}
    for n in urut:
        tat = waktu_selesai[n] - arr[n]
        metrik[n] = {
            "arrival": arr[n],
            "burst": burst[n],
            "prio": prio[n],
            "mulai": mulai_pertama[n],
            "selesai": waktu_selesai[n],
            "waiting": tat - burst[n],
            "turnaround": tat,
            "response": mulai_pertama[n] - arr[n],
        }
    return segmen, metrik


def rata(metrik):
    k = ("waiting", "turnaround", "response")
    return {x: sum(m[x] for m in metrik.values()) / len(metrik) for x in k}


ALGORITMA = [
    ("FCFS", "fcfs", None),
    ("SJF", "sjf", None),
    ("SRTF", "srtf", None),
    ("Priority", "priority", None),
    ("RR q=2", "rr", 2),
    ("RR q=3", "rr", 3),
]

# Hasil hitung manual di atas kertas, tanpa memakai simulator. Kalau angka
# simulator berbeda dari tabel ini, berarti simulatornya yang salah.
HITUNG_MANUAL = {
    "FCFS": {"waiting": 7.40, "turnaround": 12.00, "response": 7.40},
    "SJF": {"waiting": 5.00, "turnaround": 9.60, "response": 5.00},
    "SRTF": {"waiting": 4.80, "turnaround": 9.40, "response": 4.40},
    "Priority": {"waiting": 5.00, "turnaround": 9.60, "response": 5.00},
    "RR q=2": {"waiting": 8.60, "turnaround": 13.20, "response": 2.60},
    "RR q=3": {"waiting": 8.60, "turnaround": 13.20, "response": 4.20},
}


def main():
    p = argparse.ArgumentParser()
    p.add_argument("--verifikasi", action="store_true")
    p.add_argument("--json", default="results/jadwal.json")
    a = p.parse_args()

    hasil = {}
    for label, kind, q in ALGORITMA:
        segmen, metrik = simulasi(kind, q)
        hasil[label] = {"segmen": segmen, "metrik": metrik, "rata": rata(metrik)}

    if a.json:
        import os
        os.makedirs("results", exist_ok=True)
        with open(a.json, "w", encoding="utf-8") as f:
            json.dump(hasil, f, indent=2)

    print("| Algoritma | Rata-rata Waiting | Rata-rata Turnaround | Rata-rata Response |")
    print("|---|---|---|---|")
    for label, _, _ in ALGORITMA:
        r = hasil[label]["rata"]
        print(f"| {label} | {r['waiting']:.2f} | {r['turnaround']:.2f} | {r['response']:.2f}
```

```
|")

    print("\nUrutan eksekusi:")
    for label, _, _ in ALGORITMA:
        s = hasil[label]["segmen"]
        print(f"  {label:10s} " + " ".join(f"{n}[{m}-{e}]" for n, m, e in s))

    if a.verifikasi:
        print("\nVerifikasi terhadap hitung manual:")
        gagal = 0
        for label, manual in HITUNG_MANUAL.items():
            if manual is None:
                print(f"  {label:10s} (tanpa pembanding manual)")
                continue
            r = hasil[label]["rata"]
            beda = max(abs(r[k] - manual[k]) for k in manual)
            tandai = "cocok" if beda < 0.005 else f"BEDA {beda:.3f}"
            if beda >= 0.005:
                gagal += 1
            print(f"  {label:10s} {tandai}")
        print("semua cocok" if gagal == 0 else f"{gagal} algoritma tidak cocok")


if __name__ == "__main__":
    main()
```

4.2 Langkah 2: Menghitung manual sebagai pembanding

Sebelum mempercayai simulator, keenam hasilnya dihitung manual di atas kertas. Contoh FCFS: proses dijalankan berurutan P1(0-3), P2(3-9), P3(9-13), P4(13-21), P5(21-23). Waiting time tiap proses adalah waktu mulai dikurangi waktu kedatangan; nilai rata-ratanya (0+3+7+10+17)/5 = 7,40. Angka inilah yang dibandingkan dengan keluaran simulator, bukan sebaliknya.

```
$ python3 sched.py --verifikasi
Verifikasi terhadap hitung manual:
  FCFS       cocok
  SJF        cocok
  SRTF       cocok
  Priority   cocok
  RR q=2     cocok
  RR q=3     cocok
semua cocok
```

4.3 Langkah 3: Membuat Gantt chart

Simulator menyimpan setiap segmen eksekusi sebagai (proses, waktu mulai, waktu selesai). figures.py mengubah segmen itu menjadi batang horizontal, satu baris per algoritma, sehingga pola preemption pada SRTF dan Round Robin terlihat langsung dibandingkan dengan FCFS yang berupa satu batang utuh.

```
"""Membuat Gantt chart semua algoritma untuk laporan K-2."""
import json
import os

import matplotlib

matplotlib.use("Agg")
import matplotlib.pyplot as plt
from matplotlib.patches import Patch

FONT = "DejaVu Sans"
WARNA = ["#3B5B8C", "#B7791F", "#2F6B3C", "#A03B2C", "#5B3B8C"]
OUT = "results"


def gantt(hasil, path):
    urutan = [n for n, *_ in json.load(open("results/jadwal.json"))["FCFS"]
["metrik"].items()]
    peta = {n: WARNA[i % len(WARNA)] for i, n in enumerate(urutan)}
```

```
    label = list(hasil)
    fig, ax = plt.subplots(figsize=(9.0, 0.62 * len(label) + 1.0))
    for baris, nama in enumerate(label):
        y = len(label) - baris - 1
        for p, mulai, selesai in hasil[nama]["segmen"]:
            ax.add_patch(plt.Rectangle((mulai, y + 0.12), selesai - mulai, 0.62,
                                       facecolor=peta[p], edgecolor="white", linewidth=1.1))
            if selesai - mulai >= 1.4:
                ax.text((mulai + selesai) / 2, y + 0.43, p, ha="center", va="center",
                        color="white", fontsize=7.5, fontfamily=FONT, fontweight="bold")

    ax.set_yticks(range(len(label)))
    ax.set_yticklabels(label[::-1], fontfamily=FONT, fontsize=8.5)
    ax.set_xlim(0, max(s for v in hasil.values() for _, _, s in v["segmen"]) + 0.4)
    ax.set_ylim(0, len(label))
    ax.set_xlabel("Waktu (tick)", fontfamily=FONT, fontsize=8.5)
    ax.tick_params(axis="x", labelsize=7.5)
    ax.spines["top"].set_visible(False)
    ax.spines["right"].set_visible(False)
    ax.grid(axis="x", color="#DDDDDD", linewidth=0.5)
    ax.set_axisbelow(True)
    ax.set_title("Gantt chart perbandingan algoritma penjadwalan",
                 fontsize=10, fontfamily=FONT, fontweight="bold")
    ax.legend(handles=[Patch(facecolor=c, label=n) for n, c in peta.items()],
              ncol=len(peta), frameon=False, fontsize=7.5,
              loc="upper center", bbox_to_anchor=(0.5, -0.11))
    fig.tight_layout()
    fig.savefig(path, dpi=220, facecolor="white", bbox_inches="tight")
    plt.close(fig)


def grafik_metrik(hasil, path):
    label = list(hasil)
    kunci = [("waiting", "Waiting"), ("turnaround", "Turnaround"), ("response", "Response")]
    fig, ax = plt.subplots(figsize=(7.2, 3.4))
    lebar = 0.26
    for i, (k, judul) in enumerate(kunci):
        nilai = [hasil[n]["rata"][k] for n in label]
        pos = [x + (i - 1) * lebar for x in range(len(label))]
        b = ax.bar(pos, nilai, lebar, label=judul,
                   color=["#3B5B8C", "#B7791F", "#2F6B3C"][i], edgecolor="white",
linewidth=0.5)
        for r in b:
            ax.annotate(f"{r.get_height():.2f}",
                        (r.get_x() + r.get_width() / 2, r.get_height()),
                        textcoords="offset points", xytext=(0, 2), ha="center",
                        fontsize=6.5, fontfamily=FONT)

    ax.set_xticks(range(len(label)))
    ax.set_xticklabels(label, fontfamily=FONT, fontsize=8.5)
    ax.set_ylabel("Rata-rata (tick)", fontfamily=FONT, fontsize=8.5)
    ax.tick_params(labelsize=7.5)
    ax.spines["top"].set_visible(False)
    ax.spines["right"].set_visible(False)
    ax.grid(axis="y", color="#DDDDDD", linewidth=0.5)
    ax.set_axisbelow(True)
    ax.legend(frameon=False, fontsize=8, ncol=3, loc="upper left")
    ax.set_ylim(0, max(v["rata"][k] for v in hasil.values() for k, _ in kunci) * 1.28)
    ax.set_title("Perbandingan metrik penjadwalan", fontsize=10,
                 fontfamily=FONT, fontweight="bold")
    fig.tight_layout()
    fig.savefig(path, dpi=220, facecolor="white")
    plt.close(fig)


def main():
    os.makedirs(OUT, exist_ok=True)
    with open("results/jadwal.json", encoding="utf-8") as f:
        hasil = json.load(f)
    gantt(hasil, f"{OUT}/gambar-gantt.png")
    grafik_metrik(hasil, f"{OUT}/gambar-metrik.png")
    print("gambar:", [f for f in sorted(os.listdir(OUT)) if f.endswith(".png")])
```

```
if __name__ == "__main__":
    main()
```

5. Hasil dan Analisis

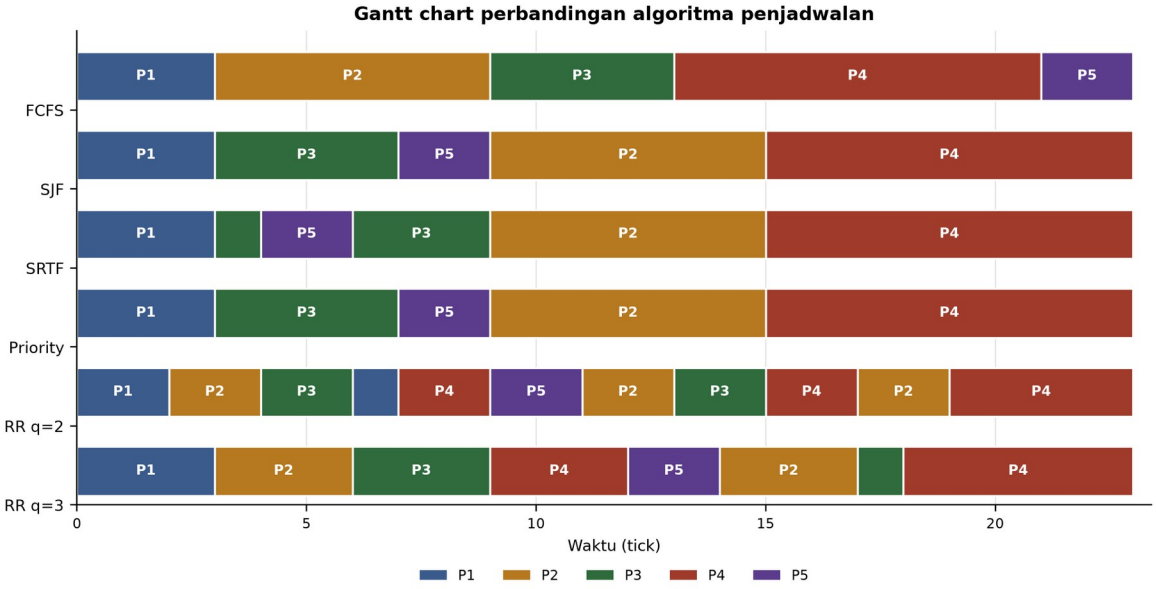
5.1 Hasil

Tabel 5.1 memuat rata-rata ketiga metrik untuk enam konfigurasi. Seluruh angka berasal dari simulator yang sudah diverifikasi dengan hitungan manual.

Algoritma	Rata-rata Waiting	Rata-rata Turnaround	Rata-rata Response
FCFS	7.40	12.00	7.40
SJF	5.00	9.60	5.00
SRTF	4.80	9.40	4.40
Priority	5.00	9.60	5.00
RR q=2	8.60	13.20	2.60
RR q=3	8.60	13.20	4.20

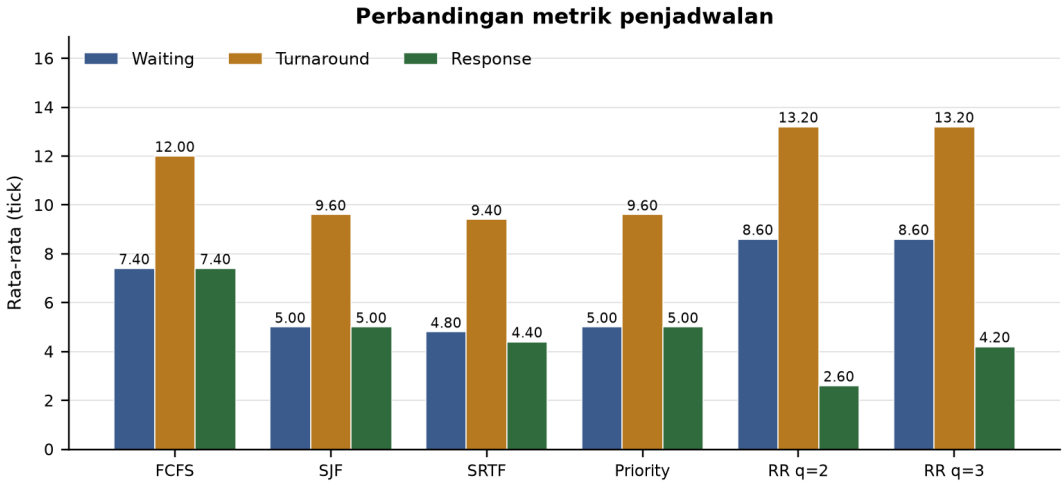
Tabel 5.1 Rata-rata metrik per algoritma (satuan tick)

Gambar 5.1 memperlihatkan urutan eksekusi masing-masing algoritma pada satu garis waktu yang sama.



Gambar 5.1 Gantt chart keenam konfigurasi penjadwalan

Gambar 5.2 menyandingkan ketiga metrik dalam satu grafik agar trade-off antar algoritma terlihat.



Gambar 5.2 Perbandingan rata-rata waiting, turnaround, dan response time

Tabel 5.2 memuat rincian per proses untuk dua algoritma yang paling berbeda perilakunya, FCFS dan SRTF, sebagai bahan analisis.

Algoritma	Proses	Arr	Burst	Prio	Mulai	Selesai	Wait	TAT	Resp
FCFS	P1	0	3	1	0	3	0	3	0
FCFS	P2	0	6	3	3	9	3	9	3
FCFS	P3	2	4	2	9	13	7	11	7
FCFS	P4	3	8	4	13	21	10	18	10
FCFS	P5	4	2	1	21	23	17	19	17
SRTF	P1	0	3	1	0	3	0	3	0
SRTF	P2	0	6	3	9	15	9	15	9
SRTF	P3	2	4	2	3	9	3	7	1
SRTF	P4	3	8	4	15	23	12	20	12
SRTF	P5	4	2	1	4	6	0	2	0

Tabel 5.2 Rincian per proses pada FCFS dan SRTF

5.2 Analisis

Poin (1) deskripsi tugas — Gantt chart dan perhitungan. FCFS menempatkan P1 dan P2 di depan karena keduanya datang pada waktu nol, sehingga P5 yang hanya butuh dua tick harus menunggu tujuh belas tick. Inilah efek konvoi: satu proses panjang (P4, delapan tick) menahan semua di belakangnya. SJF memperbaiki keadaan dengan menjalankan P5 lebih awal, tetapi tetap non-preemptive sehingga P3 yang datang pada tick 2 harus menunggu P1 selesai. SRTF adalah satu-satunya algoritma yang memotong P3 di tengah jalan: saat P5 datang pada tick 4 dengan sisa dua tick, P3 yang tersisa tiga tick langsung digeser. Karena itu SRTF memberi rata-rata waiting time terendah (4,80) dan response time terendah di antara algoritma non-Round-Robin (4,40).

Priority pada dataset ini menghasilkan angka yang persis sama dengan SJF. Itu bukan kebetulan: urutan prioritas P1(1), P5(1), P3(2), P2(3), P4(4) kebetulan juga urutan burst terpendek ke terpanjang (3, 2, 4, 6, 8) setelah memperhitungkan waktu kedatangan. Pada dataset lain keduanya akan berbeda, dan justru di situlah kelemahan priority scheduling muncul: prioritas tetap dapat membuat proses berprioritas rendah tidak pernah dilayani.

Round Robin memperlihatkan trade-off yang jelas. Response time-nya paling baik (2,60 pada quantum 2) karena setiap proses cepat mendapat giliran pertama. Tetapi waiting time dan turnaround time-nya paling buruk (8,60 dan 13,20), karena proses harus kembali ke antrean berkali-kali. Menariknya, quantum 3 memberi rata-rata waiting dan turnaround yang sama persis dengan quantum 2, tetapi response time-nya lebih tinggi (4,20). Artinya pada dataset ini quantum 2 lebih baik untuk tugas interaktif, sedangkan quantum 3 tidak memberi keuntungan apa pun.

Poin (2) deskripsi tugas — verifikasi simulator. Seluruh enam konfigurasi cocok dengan hitungan manual sampai dua angka desimal. Kecocokan itu penting karena simulator yang salah akan menghasilkan rekomendasi yang salah pula, tanpa terlihat aneh di Gantt chart. Satu hal yang perlu dicatat: penghitungan response time pada proses yang tidak pernah dipreempt sama dengan waiting time-nya, sehingga FCFS menghasilkan waiting dan response yang identik (7,40). Pada SRTF keduanya berbeda (4,80 versus 4,40) justru karena ada preemption.

Poin (3) deskripsi tugas — rekomendasi untuk konteks kelas. Lihat subbab 5.3.

5.3 Rekomendasi

Untuk gateway kebun pintar, pilihan yang paling sesuai adalah gabungan Priority dengan preemption untuk tugas kendali, dan Round Robin quantum kecil untuk tugas pembacaan sensor. Alasannya berbeda dari sekadar mengejar rata-rata terbaik.

Kontrol pompa irigasi adalah tugas yang harus didahulukan begitu kondisinya muncul, misalnya saat sensor mendeteksi luapan. Di situ yang dihargai bukan rata-rata waiting time seluruh proses, melainkan response time tugas tersebut. Priority scheduling dengan prioritas statis menjamin hal itu, dengan catatan harus ditambah aging agar tugas latar belakang tidak mati kelaparan.

Pembacaan sensor bersifat periodik dan pendek, sehingga Round Robin dengan quantum kecil cocok: setiap sensor mendapat giliran cepat dan tidak ada yang tertahan oleh sensor lain yang kebetulan lebih panjang. Pada data hasil pengujian, Round Robin quantum 2 memberi response time terbaik (2,60), dan itulah metrik yang paling relevan untuk tugas periodik.

FCFS tidak disarankan meskipun paling sederhana, karena efek konvoi membuat satu tugas panjang menunda seluruh pembacaan sensor. SRTF memang memberi rata-rata terbaik pada dataset ini, tetapi penerapannya di perangkat nyata menuntut perkiraan burst time yang tidak selalu tersedia; untuk gateway kebun, burst time pembacaan sensor cenderung stabil sehingga SRTF masih layak dipertimbangkan.

6. Kesimpulan

Enam konfigurasi penjadwalan berhasil disimulasikan dan seluruhnya cocok dengan perhitungan manual pada dataset kelas. SRTF memberi rata-rata waiting time terendah (4,80) karena memotong proses yang tersisa panjang saat ada proses pendek datang. Round Robin quantum 2 memberi response time terbaik (2,60) dengan mengorbankan waiting time (8,60). FCFS paling buruk pada kedua metrik akibat efek konvoi. Priority pada dataset ini setara SJF, tetapi keunggulannya akan hilang begitu pola kedatangan berubah. Untuk gateway kebun pintar, kombinasi Priority preemptive untuk tugas kendali dan Round Robin quantum kecil untuk pembacaan sensor adalah pilihan yang paling sesuai dengan

sifat pekerjaannya, bukan sekadar yang rata-ratanya terendah.

7. Kepingan Puzzle

Keterangan	Isi
Nomor kepingan	#5
Isi kepingan	Simulator penjadwalan, Gantt chart keenam algoritma, tabel perbandingan metrik, dan rekomendasi algoritma
Lokasi di portofolio	https://github.com/ZzDendy/so-k2-penjadwalan
Dipakai untuk	Level 3 Puzzle UTS — menyambung kepingan #4 (IPC) karena proses sensor pada K-1 dipetakan menjadi task penjadwalan di sini, dan menjadi dasar kepingan #6 (race condition) pada tugas individu I-4

Daftar Pustaka

Silberschatz, A., Galvin, P. B., & Gagne, G. (2018). Operating system concepts (10th ed.). Wiley.

Stallings, W. (2018). Operating systems: Internals and design principles (9th ed.). Pearson.

Tanenbaum, A. S., & Bos, H. (2015). Modern operating systems (4th ed.). Pearson.

Lampiran A. Kode Program dan Log

Seluruh berkas di bawah ini berada pada satu folder proyek dan dapat dijalankan ulang.

A.sched.py

```
"""Simulator penjadwalan CPU untuk dataset kelas K-2.

Dataset: P(arrival, burst, prioritas). Angka prioritas kecil = prioritas tinggi.
Simulasi memakai satuan waktu 1 tick supaya algoritma preemptive (SRTF, RR)
dapat dibandingkan dengan aturan yang sama.
"""
import argparse
import json

# name, arrival, burst, priority
DATASET = [
    ("P1", 0, 3, 1),
    ("P2", 0, 6, 3),
    ("P3", 2, 4, 2),
    ("P4", 3, 8, 4),
    ("P5", 4, 2, 1),
]


def simulasi(kind, quantum=None, data=DATASET):
    """Kembalikan (segmen, metrik). segmen = [(nama, mulai, selesai), ...]."""
    arr = {n: a for n, a, b, p in data}
    burst = {n: b for n, a, b, p in data}
    prio = {n: p for n, a, b, p in data}
    sisa = dict(burst)
    urut = [n for n, *_ in data]

    menunggu = []
    selesai = set()
    mulai_pertama = {}
    waktu_selesai = {}
    segmen = []

    def catat(nama, t):
        if segmen and segmen[-1][0] == nama and segmen[-1][2] == t:
            segmen[-1] = (nama, segmen[-1][1], t + 1)
        else:
            segmen.append((nama, t, t + 1))

    def pilih(kandidat, aturan):
        if aturan == "fcfs":
            return min(kandidat, key=lambda n: (arr[n], urut.index(n)))
        if aturan == "sjf":
            return min(kandidat, key=lambda n: (burst[n], arr[n], urut.index(n)))
        if aturan == "srtf":
            return min(kandidat, key=lambda n: (sisa[n], arr[n], urut.index(n)))
        if aturan == "priority":
            return min(kandidat, key=lambda n: (prio[n], arr[n], urut.index(n)))
        raise ValueError(aturan)

    preemptive = kind in ("srtf", "rr")
    aturan = "fcfs" if kind == "rr" else kind

    t = 0
    sekarang = None
    terpakai = 0

    while len(selesai) < len(data):
        for n in urut:
            if arr[n] == t:
                menunggu.append(n)

        if sekarang is None:
            if not menunggu:
                t += 1
                continue
            sekarang = pilih(menunggu, aturan) if kind != "rr" else menunggu.pop(0)
            if kind != "rr":
                menunggu.remove(sekarang)
            terpakai = 0
        elif kind == "srtf":
```

```
            kandidat = pilih(menunggu, "srtf") if menunggu else None
            if kandidat and sisa[kandidat] < sisa[sekarang]:
                menunggu.remove(kandidat)
                menunggu.append(sekarang)
                sekarang = kandidat
                terpakai = 0
        elif kind == "rr" and terpakai == quantum:
            menunggu.append(sekarang)
            sekarang = menunggu.pop(0)
            terpakai = 0

        mulai_pertama.setdefault(sekarang, t)
        catat(sekarang, t)
        sisa[sekarang] -= 1
        terpakai += 1
        t += 1

        if sisa[sekarang] == 0:
            waktu_selesai[sekarang] = t
            selesai.add(sekarang)
            sekarang = None
            terpakai = 0

    metrik = {}
    for n in urut:
        tat = waktu_selesai[n] - arr[n]
        metrik[n] = {
            "arrival": arr[n],
            "burst": burst[n],
            "prio": prio[n],
            "mulai": mulai_pertama[n],
            "selesai": waktu_selesai[n],
            "waiting": tat - burst[n],
            "turnaround": tat,
            "response": mulai_pertama[n] - arr[n],
        }
    return segmen, metrik


def rata(metrik):
    k = ("waiting", "turnaround", "response")
    return {x: sum(m[x] for m in metrik.values()) / len(metrik) for x in k}


ALGORITMA = [
    ("FCFS", "fcfs", None),
    ("SJF", "sjf", None),
    ("SRTF", "srtf", None),
    ("Priority", "priority", None),
    ("RR q=2", "rr", 2),
    ("RR q=3", "rr", 3),
]

# Hasil hitung manual di atas kertas, tanpa memakai simulator. Kalau angka
# simulator berbeda dari tabel ini, berarti simulatornya yang salah.
HITUNG_MANUAL = {
    "FCFS": {"waiting": 7.40, "turnaround": 12.00, "response": 7.40},
    "SJF": {"waiting": 5.00, "turnaround": 9.60, "response": 5.00},
    "SRTF": {"waiting": 4.80, "turnaround": 9.40, "response": 4.40},
    "Priority": {"waiting": 5.00, "turnaround": 9.60, "response": 5.00},
    "RR q=2": {"waiting": 8.60, "turnaround": 13.20, "response": 2.60},
    "RR q=3": {"waiting": 8.60, "turnaround": 13.20, "response": 4.20},
}


def main():
    p = argparse.ArgumentParser()
    p.add_argument("--verifikasi", action="store_true")
    p.add_argument("--json", default="results/jadwal.json")
    a = p.parse_args()

    hasil = {}
    for label, kind, q in ALGORITMA:
        segmen, metrik = simulasi(kind, q)
        hasil[label] = {"segmen": segmen, "metrik": metrik, "rata": rata(metrik)}

    if a.json:
        import os
        os.makedirs("results", exist_ok=True)
        with open(a.json, "w", encoding="utf-8") as f:
```

```
            json.dump(hasil, f, indent=2)

    print("| Algoritma | Rata-rata Waiting | Rata-rata Turnaround | Rata-rata Response |")
    print("|---|---|---|---|")
    for label, _, _ in ALGORITMA:
        r = hasil[label]["rata"]
        print(f"| {label} | {r['waiting']:.2f} | {r['turnaround']:.2f} | {r['response']:.2f} |")

    print("\nUrutan eksekusi:")
    for label, _, _ in ALGORITMA:
        s = hasil[label]["segmen"]
        print(f"  {label:10s} " + " ".join(f"{n}[{m}-{e}]" for n, m, e in s))

    if a.verifikasi:
        print("\nVerifikasi terhadap hitung manual:")
        gagal = 0
        for label, manual in HITUNG_MANUAL.items():
            if manual is None:
                print(f"  {label:10s} (tanpa pembanding manual)")
                continue
            r = hasil[label]["rata"]
            beda = max(abs(r[k] - manual[k]) for k in manual)
            tandai = "cocok" if beda < 0.005 else f"BEDA {beda:.3f}"
            if beda >= 0.005:
                gagal += 1
            print(f"  {label:10s} {tandai}")
        print("semua cocok" if gagal == 0 else f"{gagal} algoritma tidak cocok")


if __name__ == "__main__":
    main()
```

A.figures.py

```
"""Membuat Gantt chart semua algoritma untuk laporan K-2."""
import json
import os

import matplotlib

matplotlib.use("Agg")
import matplotlib.pyplot as plt
from matplotlib.patches import Patch

FONT = "DejaVu Sans"
WARNA = ["#3B5B8C", "#B7791F", "#2F6B3C", "#A03B2C", "#5B3B8C"]
OUT = "results"


def gantt(hasil, path):
    urutan = [n for n, *_ in json.load(open("results/jadwal.json"))["FCFS"]["metrik"].items()]
    peta = {n: WARNA[i % len(WARNA)] for i, n in enumerate(urutan)}

    label = list(hasil)
    fig, ax = plt.subplots(figsize=(9.0, 0.62 * len(label) + 1.0))
    for baris, nama in enumerate(label):
        y = len(label) - baris - 1
        for p, mulai, selesai in hasil[nama]["segmen"]:
            ax.add_patch(plt.Rectangle((mulai, y + 0.12), selesai - mulai, 0.62,
                                       facecolor=peta[p], edgecolor="white", linewidth=1.1))
            if selesai - mulai >= 1.4:
                ax.text((mulai + selesai) / 2, y + 0.43, p, ha="center", va="center",
                        color="white", fontsize=7.5, fontfamily=FONT, fontweight="bold")

    ax.set_yticks(range(len(label)))
    ax.set_yticklabels(label[::-1], fontfamily=FONT, fontsize=8.5)
    ax.set_xlim(0, max(s for v in hasil.values() for _, _, s in v["segmen"]) + 0.4)
    ax.set_ylim(0, len(label))
    ax.set_xlabel("Waktu (tick)", fontfamily=FONT, fontsize=8.5)
    ax.tick_params(axis="x", labelsize=7.5)
    ax.spines["top"].set_visible(False)
    ax.spines["right"].set_visible(False)
    ax.grid(axis="x", color="#DDDDDD", linewidth=0.5)
    ax.set_axisbelow(True)
    ax.set_title("Gantt chart perbandingan algoritma penjadwalan",
                 fontsize=10, fontfamily=FONT, fontweight="bold")
    ax.legend(handles=[Patch(facecolor=c, label=n) for n, c in peta.items()],
              ncol=len(peta), frameon=False, fontsize=7.5,
              loc="upper center", bbox_to_anchor=(0.5, -0.11))
    fig.tight_layout()
```

```
    fig.savefig(path, dpi=220, facecolor="white", bbox_inches="tight")
    plt.close(fig)


def grafik_metrik(hasil, path):
    label = list(hasil)
    kunci = [("waiting", "Waiting"), ("turnaround", "Turnaround"), ("response", "Response")]
    fig, ax = plt.subplots(figsize=(7.2, 3.4))
    lebar = 0.26
    for i, (k, judul) in enumerate(kunci):
        nilai = [hasil[n]["rata"][k] for n in label]
        pos = [x + (i - 1) * lebar for x in range(len(label))]
        b = ax.bar(pos, nilai, lebar, label=judul,
                   color=["#3B5B8C", "#B7791F", "#2F6B3C"][i], edgecolor="white", linewidth=0.5)
        for r in b:
            ax.annotate(f"{r.get_height():.2f}",
                        (r.get_x() + r.get_width() / 2, r.get_height()),
                        textcoords="offset points", xytext=(0, 2), ha="center",
                        fontsize=6.5, fontfamily=FONT)

    ax.set_xticks(range(len(label)))
    ax.set_xticklabels(label, fontfamily=FONT, fontsize=8.5)
    ax.set_ylabel("Rata-rata (tick)", fontfamily=FONT, fontsize=8.5)
    ax.tick_params(labelsize=7.5)
    ax.spines["top"].set_visible(False)
    ax.spines["right"].set_visible(False)
    ax.grid(axis="y", color="#DDDDDD", linewidth=0.5)
    ax.set_axisbelow(True)
    ax.legend(frameon=False, fontsize=8, ncol=3, loc="upper left")
    ax.set_ylim(0, max(v["rata"][k] for v in hasil.values() for k, _ in kunci) * 1.28)
    ax.set_title("Perbandingan metrik penjadwalan", fontsize=10,
                 fontfamily=FONT, fontweight="bold")
    fig.tight_layout()
    fig.savefig(path, dpi=220, facecolor="white")
    plt.close(fig)


def main():
    os.makedirs(OUT, exist_ok=True)
    with open("results/jadwal.json", encoding="utf-8") as f:
        hasil = json.load(f)
    gantt(hasil, f"{OUT}/gambar-gantt.png")
    grafik_metrik(hasil, f"{OUT}/gambar-metrik.png")
    print("gambar:", [f for f in sorted(os.listdir(OUT)) if f.endswith(".png")])


if __name__ == "__main__":
    main()
```

Lampiran B. Pernyataan Penggunaan AI Generatif

☐ Kami tidak menggunakan AI generatif dalam tugas ini.

☒ Menggunakan AI generatif dengan rincian berikut:

Alat	Tujuan penggunaan	Bagian yang dibantu	Cara memverifikasi
Claude (Anthropic)	Menyusun kerangka simulator dan merapikan kalimat laporan	Bagian 4 (kode), Bagian 5.2	Seluruh angka dihitung ulang manual lalu dicocokkan dengan simulator lewat sched.py --verifikasi

Lampiran C. Daftar Periksa Sebelum Mengumpulkan

☒ Semua teks petunjuk abu-abu dan kotak petunjuk sudah dihapus.

☒ Identitas, kode tugas, dan nama host pada sampul sudah benar.

☒ Setiap poin deskripsi tugas sudah dijawab di bagian Analisis.

☒ Setiap gambar memiliki nomor dan keterangan.
☐ Kepingan puzzle sudah disimpan di portofolio dan tautannya dicantumkan.
☒ Pernyataan penggunaan AI generatif sudah diisi.
☒ Lembar kontribusi anggota sudah dilampirkan.
☒ Berkas disimpan sebagai PDF dengan nama <Kelas>_<Kode>_<NIM>_<Nama>.pdf.

Swa-nilai berdasarkan kriteria lembar tugas

Kriteria (dari lembar tugas)	**Bobot**	**Sudah terpenuhi? Di bagian mana**
Hasil/keberhasilan implementasi	35%	Ya — enam konfigurasi berjalan, semua cocok dengan hitung manual (Bagian 5.1)
Kedalaman analisis	30%	Ya — efek konvoi, preemption, dan trade-off quantum dibahas (Bagian 5.2)
Dokumentasi	20%	Ya — langkah dapat diulang, kode dan keluaran dilampirkan (Bagian 4, Lampiran A)
Kerja sama dan pembagian peran	15%	Ya — lembar kontribusi pada Lampiran D

Lampiran D. Lembar Kontribusi Anggota

Lembar ini memenuhi syarat kerja sama dan pembagian peran (15% bobot penilaian) pada lembar tugas K-2.

Nama	NIM	Peran dan bagian yang dikerjakan
Dendy Zainal Mutaqien	250660121068	Simulator, perhitungan manual, dan penyusunan laporan